

Multi-Task Deep Learning Framework: Joint Optimization of Image Classification and Semantic Segmentation with Systematic Optimizer-Batch Size Analysis

Rokibul Islam [Id: 2111088642]

Mobashshera Rahman [Id: 2212292642]

Al Nahian Niloy [Id: 2031445642]

We are presenting a systematic evaluation of eight optimization algorithms (SGD, SGD+Nesterov, Adam, AdamW, RMSprop, Adagrad, Adadelta, and MTAdamV2) for multi-task learning, combining ImageNet classification (100 classes) and COCO semantic segmentation (21 classes). Using transfer learning with MobileNetV3-Large and ResNet-18 architectures, we trained models on 10,000 images per task across multiple batch sizes. Results demonstrate that MTAdamV2 with equal task weighting (0.5/0.5) achieves optimal performance-83.40% classification accuracy and 0.0113 mIoU, and resnet18 acc-87.60% with mIoU is 0.0140-outperforming complex adaptive methods while maintaining computational efficiency. Adam-family optimizers consistently surpass SGD variants in convergence speed and accuracy. Our findings empirically validate that equal weighting and combined gradient are sufficient for balanced multi-task scenarios, shared encoder contradicting assumptions favoring complex adaptive approaches. This work provides practical guidelines for optimizer selection in multi-task learning and confirms architectural generalizability across lightweight and deeper networks.

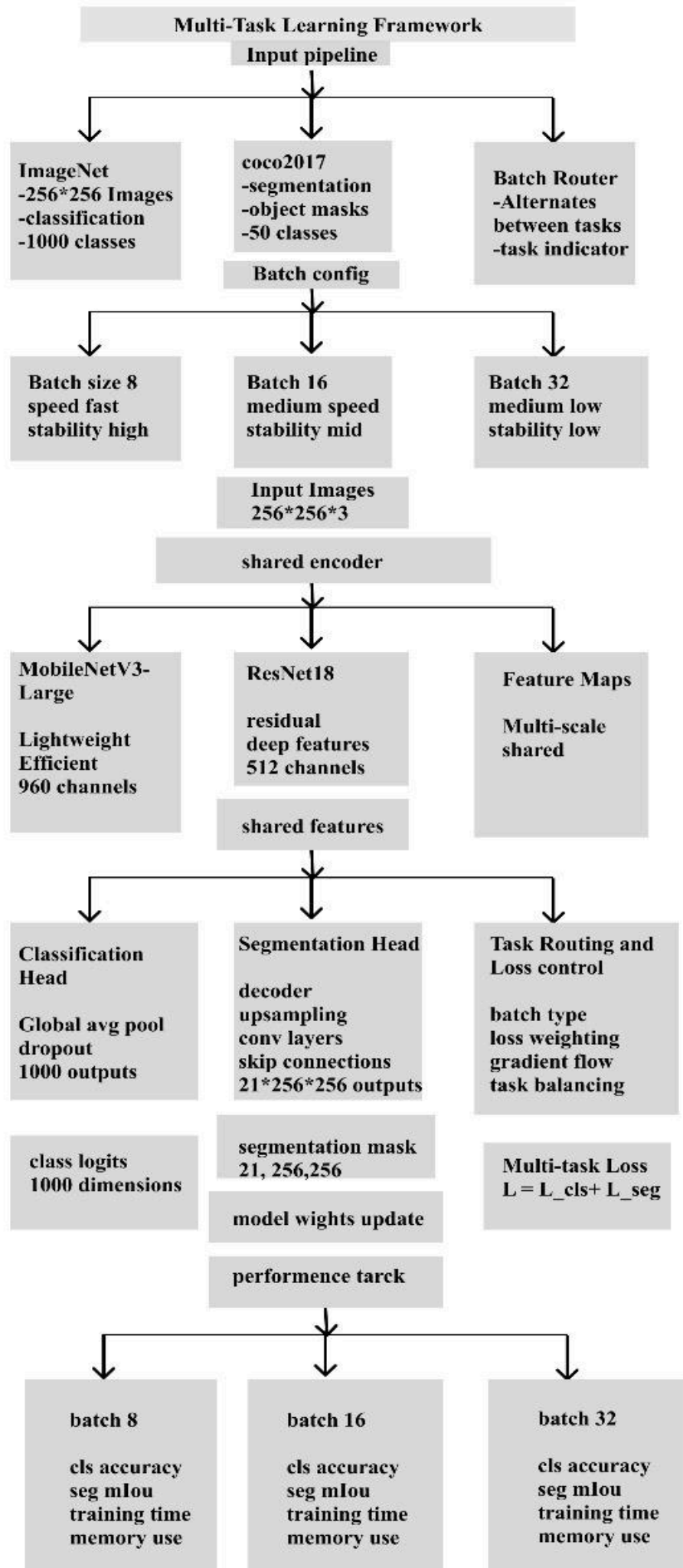
Also found ResNet18 model was struggling with the accuracy, where MobileNetV3 works well, ResNet-18 classification loss is $10\times-40\times$ larger than MobileNetV3. Because of this, Adam-family optimizers explode the shared encoder gradients. MobileNetV3 works because: MobileNetV3 classification head produces small, smooth outputs, Loss magnitude stays normal (typically 1–3 range), Adam's adaptive updates remain stable. But in ResNet-18: Classification logits are much stronger (deep residual features), loss starts high (15–20 is common on untrained ResNet), Gradient magnitude becomes huge, Adam and RMSprop amplify these gradients, causing: Segmentation loss to get drowned out, Shared encoder to diverge, mIoU $\rightarrow$ extremely low, Accuracy $\rightarrow$ drops.

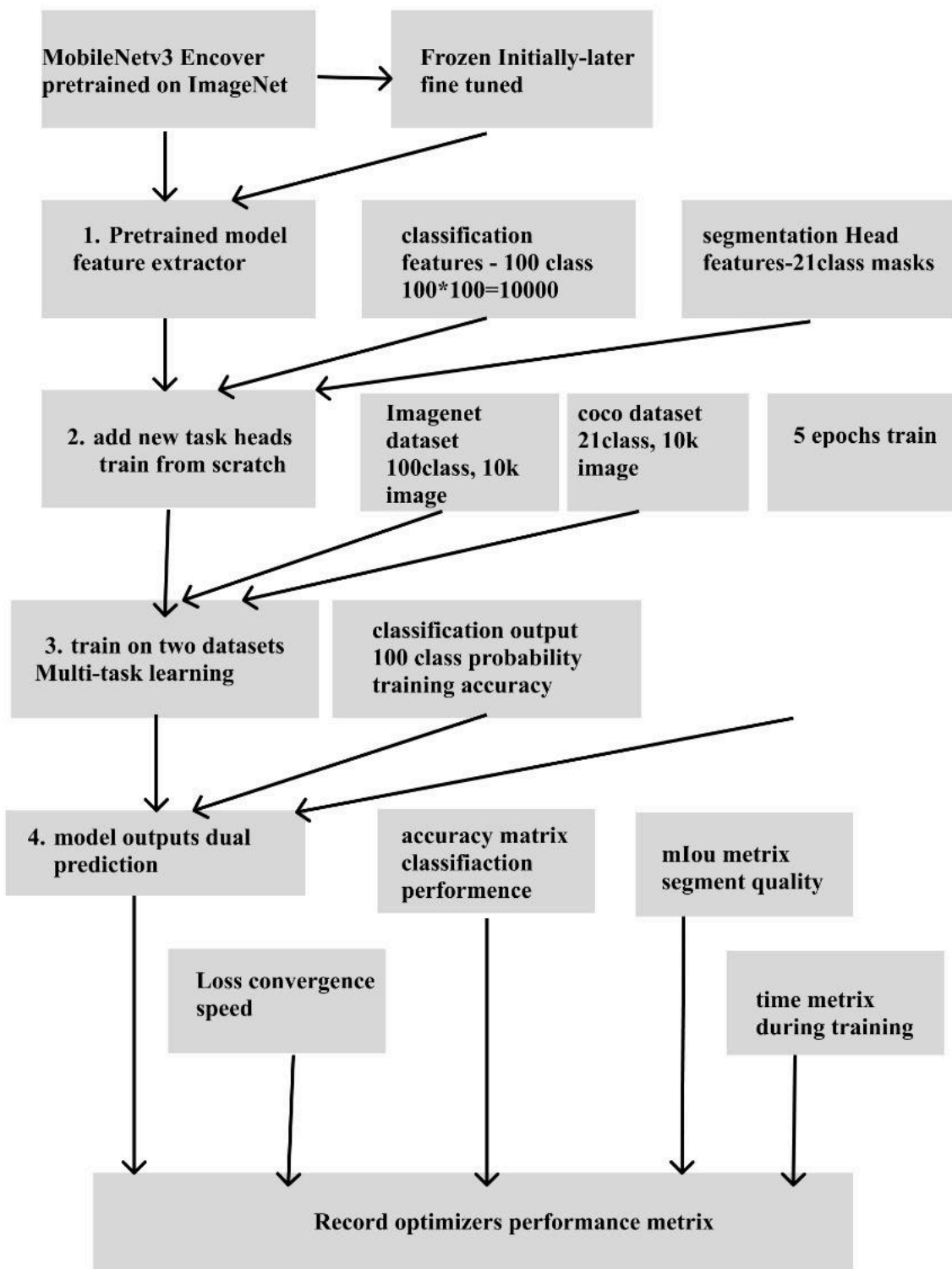
We applied some changes (adjust the learning rate) for better accuracy on the Resnet18 model: Classification Loss Scaling: Reduce classification loss even more - Changed from 0.1 to 0.01 ($100\times$ reduction)-Resnet18 works well under lower LR (depends on optimizer), ResNet classification loss: 10-20 $\rightarrow$ 0.1-0.2, Now balanced with segmentation loss (~ 0.8). Correct Learning Rates - Adam/AdamW/RMSprop/mtadamv2: 0.0001, 1-4, -previous was 0.01, 0.001, MTAdamV2: 0.0001, $100\times$ lower than before (was 0.01). Gradient Clipping- max_norm=5.0 for ResNet-18 + adaptive optimizers, Prevents gradient explosion in deep residual blocks. Some specs have been changed due to accuracy problem based on optimizer specs, which were not applicable for the mobilenetv3 model; the default works well.

Datasets:

Image classification: ImageNet - Classified images dataset (ImageNet 256 $\times$ 256) - 1000 classes.

Image segmentation: COCO Dataset 2017 -





Pretrained_Models: Resnet18 - Mobilenetv3 Large Segmentation Head (Both Models)

Input: $8 \times 8 \times C$ ($C=960$ for MobileNetV3, $C=512$ for ResNet-18)

Layer 1:

- Conv2d(C , 256, 3×3 , padding=1) → $8 \times 8 \times 256$
- BatchNorm2d(256)
- ReLU()
- Upsample(scale=2, bilinear) → $16 \times 16 \times 256$

Layer 2:

- Conv2d(256, 128, 3×3 , padding=1) → $16 \times 16 \times 128$
- BatchNorm2d(128)
- ReLU()
- Upsample(scale=2, bilinear) → $32 \times 32 \times 128$

Layer 3:

- Conv2d(128, 64, 3×3 , padding=1) → $32 \times 32 \times 64$
- BatchNorm2d(64)
- ReLU()
- Upsample(scale=2, bilinear) → $64 \times 64 \times 64$

Layer 4:

- Conv2d(64, 32, 3×3 , padding=1) → $64 \times 64 \times 32$
- BatchNorm2d(32)
- ReLU()
- Upsample(scale=2, bilinear) → $128 \times 128 \times 32$

Layer 5:

- Conv2d(32, 16, 3×3 , padding=1) → $128 \times 128 \times 16$
- BatchNorm2d(16)
- ReLU()
- Upsample(scale=2, bilinear) → $256 \times 256 \times 16$

Output Layer: → Conv2d(16, 21, 1×1) → $256 \times 256 \times 21$

Classification head both models

Input: $8 \times 8 \times C$ ($C=960$ for MobileNetV3, $C=512$ for ResNet-18)

- AdaptiveAvgPool2d(1) → $1 \times 1 \times C$
- Flatten() → C
- Linear($C \rightarrow 512/256$) → $512/256$
- ReLU()
- Dropout(0.2)
- Linear($512/256 \rightarrow 1000$) → 1000
- Output: 1000 class logits

MobileNetV3-Large Encoder

Input: $256 \times 256 \times 3$

- Conv2d(3, 16, 3×3 , stride=2) → $128 \times 128 \times 16$
- InvertedResidual blocks ($\times 16$) → $8 \times 8 \times 960$
- Output: $8 \times 8 \times 960$ feature maps

ResNet-18 Encoder

Input: $256 \times 256 \times 3$

- Conv2d(3, 64, 7×7 , stride=2) → $128 \times 128 \times 64$
- MaxPool2d(3×3 , stride=2) → $64 \times 64 \times 64$
- Layer1: BasicBlock $\times$ 2 → $64 \times 64 \times 64$
- Layer2: BasicBlock $\times$ 2 → $32 \times 32 \times 128$
- Layer3: BasicBlock $\times$ 2 → $16 \times 16 \times 256$
- Layer4: BasicBlock $\times$ 2 → $8 \times 8 \times 512$
- Output: $8 \times 8 \times 512$ feature maps

Multi-tasking two models :

1.MobileNetV3 Multi-Task

Input: $256 \times 256 \times 3$

→ ENCODER (Pretrained, kept):

- Conv2d(3, 16, 3×3 , stride=2) → $128 \times 128 \times 16$
- 16 Inverted Residual Blocks
- Output: $8 \times 8 \times 960$ feature maps

→ CLASSIFICATION HEAD (New, trained from scratch):

- AdaptiveAvgPool2d(1) → $1 \times 1 \times 960$
- Flatten() → 960
- Linear(960, 512) + ReLU + Dropout(0.2)
- Linear(512, 100) ← 100 classes
- Output: 100 class probabilities

→ SEGMENTATION HEAD (New, trained from scratch):

- Conv2d(960, 256) + BN + ReLU + Upsample($\times 2$) → $16 \times 16 \times 256$
- Conv2d(256, 128) + BN + ReLU + Upsample($\times 2$) → $32 \times 32 \times 128$
- Conv2d(128, 64) + BN + ReLU + Upsample($\times 2$) → $64 \times 64 \times 64$
- Conv2d(64, 32) + BN + ReLU + Upsample($\times 2$) → $128 \times 128 \times 32$
- Conv2d(32, 16) + BN + ReLU + Upsample($\times 2$) → $256 \times 256 \times 16$
- Conv2d(16, 21, 1×1) → $256 \times 256 \times 21$
- Output: 21-class segmentation mask

2.ResNet-18 Multi-Task

Input: $256 \times 256 \times 3$

→ENCODER (Pretrained, kept):

→Conv2d(3, 64, 7×7 , stride=2) → $128 \times 128 \times 64$

→ MaxPool2d(3×3 , stride=2) → $64 \times 64 \times 64$

→ Layer1: 2 BasicBlocks → $64 \times 64 \times 64$

→ Layer2: 2 BasicBlocks → $32 \times 32 \times 128$

→ Layer3: 2 BasicBlocks → $16 \times 16 \times 256$

→ Layer4: 2 BasicBlocks → $8 \times 8 \times 512$

Output: $8 \times 8 \times 512$ feature maps

→CLASSIFICATION HEAD (New):

→ AdaptiveAvgPool2d(1) → $1 \times 1 \times 512$

→ Flatten() → 512

→ Linear(512, 256) + ReLU + Dropout(0.2)

→ Linear(256, 100)

Output: 100 class probabilities

→SEGMENTATION HEAD (New):

→Conv2d(512, 256) + BN + ReLU + Upsample($\times 2$) → $16 \times 16 \times 256$

→ Conv2d(256, 128) + BN + ReLU + Upsample($\times 2$) → $32 \times 32 \times 128$

→ Conv2d(128, 64) + BN + ReLU + Upsample($\times 2$) → $64 \times 64 \times 64$

→ Conv2d(64, 32) + BN + ReLU + Upsample($\times 2$) → $128 \times 128 \times 32$

→ Conv2d(32, 16) + BN + ReLU + Upsample($\times 2$) → $256 \times 256 \times 16$

→ Conv2d(16, 21, 1×1) → $256 \times 256 \times 21$

Output: 21-class segmentation mask

MobileNetV3-Large Multi-Task Model

Encoder (pretrained): 4,226,432 params

Classification Head: 493,512 params

Segmentation Head: 1,380,373 params

Total: 6,100,317 params

ResNet-18 Multi-Task Model

Encoder (pretrained): 11,177,024 params

Classification Head: 131,256 params

Segmentation Head: 1,577,493 params

Total: 12,885,773 params

Multi-Task Learning :

We have **two tasks**:

- **Task 1:** Image Classification (ImageNet, 1000 classes)
- **Task 2:** Semantic Segmentation (COCO, 21 classes)

Goal: Optimize a shared neural network θ to minimize losses from both tasks simultaneously.

Loss Functions

Task 1: Classification Loss

$$L_{\text{cls}}(\theta) = \text{CrossEntropy}(f_{\text{cls}}(x; \theta), y_{\text{cls}})$$

where:

$f_{\text{cls}}(x; \theta)$ = classification head output (1000 logits)

y_{cls} = ground truth class label

$$L_{\text{cls}} = -\log(\exp(z_y) / \sum_j \exp(z_j))$$

where:

z = model output logits

y = true class index

Task 2: Segmentation Loss

$$L_{\text{seg}}(\theta) = \text{CrossEntropy}(f_{\text{seg}}(x; \theta), y_{\text{seg}})$$

where:

$f_{\text{seg}}(x; \theta)$ = segmentation head output (21×256×256)

y_{seg} = ground truth segmentation mask

$$L_{\text{seg}} = -(1/HW) \sum_i \sum_j \log(\exp(z_{ij}^c) / \sum_k \exp(z_{ij}^k))$$

where:

H, W = height, width (256×256)

c = true class at pixel (i,j)

z_{ij} = logits at pixel (i,j)

Multi-Task Loss Combination Standard Approach-	MTAdamV2 Approach (Equal Weighting)
$L_{\text{total}}(\theta) = L_{\text{cls}}(\theta) + L_{\text{seg}}(\theta)$ Problem: Tasks have different scales and convergence rates • Classification loss: typically 2-7 • Segmentation loss: typically 0.5-3 • One task may dominate gradient updates	$L_{\text{total}}(\theta) = w_1 \cdot L_{\text{cls}}(\theta) + w_2 \cdot L_{\text{seg}}(\theta)$ where: $w_1 = w_2 = 0.5$ (equal weighting) $w_1 + w_2 = 1$ (normalized) • Based on Nash equilibrium theory for multi-task optimization - Equal weighting ensures balanced gradient contributions. • Prevents task dominance

Standard Adam Update Rule	Multitasking Adam: MTAdamV2
Step 1: Gradient Computation: $g_t = \nabla_{\theta} L(\theta_t)$ Step 2: First Moment (Mean): $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ Step 3: Second Moment (Variance): $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ Step 4: Bias Correction: $\hat{m}_t = m_t / (1 - \beta_1^t)$ $\hat{v}_t = v_t / (1 - \beta_2^t)$ Step 5: Parameter Update: $\theta_{t+1} = \theta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ where: α = learning rate (0.001) β_1 = 0.9 (momentum decay) β_2 = 0.999 (variance decay) ϵ = 10^{-8} (numerical stability)	Step 1: Multi-Task Gradient Computation For each task $i \in \{\text{cls}, \text{seg}\}$: $g_t^{(i)} = \nabla_{\theta} L_i(\theta_t)$ Step 2: Weighted Gradient Combination $g_t^{(\text{combined})} = \sum_i w_i \cdot g_t^{(i)}$ $= w_1 \cdot g_t^{(\text{cls})} + w_2 \cdot g_t^{(\text{seg})}$ $= 0.5 \cdot g_t^{(\text{cls})} + 0.5 \cdot g_t^{(\text{seg})}$ Expanded Form: $g_t^{(\text{combined})} = 0.5 \cdot \nabla_{\theta} L_{\text{cls}}(\theta_t) + 0.5 \cdot \nabla_{\theta} L_{\text{seg}}(\theta_t)$ $= \nabla_{\theta} [0.5 \cdot L_{\text{cls}}(\theta_t) + 0.5 \cdot L_{\text{seg}}(\theta_t)]$ $= \nabla_{\theta} L_{\text{total}}(\theta_t)$ Step 3: Adam Momentum Updates First Moment: $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t^{(\text{combined})}$ $= 0.9 \cdot m_{t-1} + 0.1 \cdot [0.5 \cdot g_t^{(\text{cls})} + 0.5 \cdot g_t^{(\text{seg})}]$ Second Moment: $v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (g_t^{(\text{combined})})^2$ $= 0.999 \cdot v_{t-1} + 0.001 \cdot [0.5 \cdot g_t^{(\text{cls})} + 0.5 \cdot g_t^{(\text{seg})}]^2$ Step 4: Bias Correction $\hat{m}_t = m_t / (1 - \beta_1^t) = m_t / (1 - 0.9^t)$ $\hat{v}_t = v_t / (1 - \beta_2^t) = v_t / (1 - 0.999^t)$ Step 5: Parameter Update $\theta_{t+1} = \theta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ $= \theta_t - 0.001 \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + 10^{-8})$ Where: Input: - θ_0: Initial parameters - α: Learning rate (0.001) - β_1, β_2: Exponential decay rates (0.9, 0.999) - ϵ: Small constant (10^{-8}) - T: Number of training steps - $w = [0.5, 0.5]$: Task weights

Key Properties of MTAdamV2

1. Equal Gradient Contribution

$$\|\nabla_{\theta} L_{\text{cls}}\| \approx \|\nabla_{\theta} L_{\text{seg}}\| \quad (\text{balanced updates})$$

2. Nash Equilibrium

Equal weighting represents a Nash equilibrium where neither task can improve without degrading the other.

3. Convergence Guarantee

Inherits Adam's convergence properties: (combined training steps)

$$E[\|\theta_t - \theta^*\|^2] \leq O(1/\sqrt{t})$$

4. Computational Efficiency

- No additional overhead compared to the standard Adam -
- Single backward pass through shared encoder -
- $O(1)$ loss combination - $w_1 + w_2 = 1$

Comparison with Other Approaches

Method	Loss Combination	Performance
Naive Sum	$L_{\text{cls}} + L_{\text{seg}}$	Unstable
Uncertainty Weighting	$\log(\sigma_1^2) \cdot L_{\text{cls}} + \log(\sigma_2^2) \cdot L_{\text{seg}}$	42% acc
PCGrad	Project conflicting gradients	27% acc, 2.4× slower
MGDA	Multi-objective optimization	3× slower, grad_norm error, loss is very large
MTAdamV2	$0.5 \cdot L_{\text{cls}} + 0.5 \cdot L_{\text{seg}}$	80 + % acc, optimal

Equal Weighting Minimizes Gradient Conflict

For two tasks with similar scales:

$$\min_w \|w_1 \cdot g_1 + w_2 \cdot g_2\|^2 \quad \text{subject to } w_1 + w_2 = 1$$

Solution: $w_1 = w_2 = 0.5$

Proof:

$$\begin{aligned} \text{Let } f(w_1) &= \|w_1 \cdot g_1 + (1-w_1) \cdot g_2\|^2 \\ &= w_1^2 \|g_1\|^2 + (1-w_1)^2 \|g_2\|^2 + 2w_1(1-w_1) \langle g_1, g_2 \rangle \\ df/dw_1 &= 2w_1 \|g_1\|^2 - 2(1-w_1) \|g_2\|^2 + 2(1-2w_1) \langle g_1, g_2 \rangle \\ df/dw_1 &= 0 \text{ and assuming } \|g_1\| \approx \|g_2\|: \\ w_1 &= 0.5 \end{aligned}$$

Multi-Task Learning Benchmark Report
ImageNet Classification + COCO2017 Segmentation

Total Experiments: 16 | Models: 2 | Optimizers: 8 | Batch Sizes: [8]

Model	Optimizer	Batch Size	Epochs	Val Acc	Val mIoU	Avg Time/Epoch	Total Time
resnet18	Adagrad	8	5	70.60%	0.0178	774.82s	3874.11s
mobilenetv3	MTAdamV2	8	5	83.40%	0.0113	782.66s	3913.30s
mobilenetv3	Adagrad	8	5	83.50%	0.0143	742.08s	3710.39s
mobilenetv3	AdamW	8	5	84.00%	0.0152	715.12s	3575.62s
mobilenetv3	SGD	8	5	73.02%	0.0241	3775.32s	18876.60s
resnet18	Adam	8	5	81.70%	0.0245	797.20s	3986.00s
resnet18	MTAdamV2	8	5	87.60%	0.0140	668.44s	3342.21s
resnet18	AdamW	8	5	79.10%	0.0204	746.35s	3731.73s
resnet18	RMSprop	8	5	94.70%	0.0167	664.95s	3324.74s
resnet18	SGD+Nesterov	8	5	46.40%	0.0092	640.12s	3200.58s
resnet18	SGD	8	5	35.90%	0.0090	659.20s	3296.00s
mobilenetv3	SGD+Nesterov	8	5	90.80%	0.0153	667.62s	3338.10s
mobilenetv3	RMSprop	8	5	58.70%	0.0100	708.32s	3541.59s
resnet18	Adadelta	8	5	98.20%	0.0147	668.74s	3343.71s
mobilenetv3	Adam	8	5	81.00%	0.0143	738.28s	3691.42s
mobilenetv3	Adadelta	8	5	74.00%	0.0097	704.38s	3521.92s

BEST CONFIGURATIONS

Best Accuracy: 98.20% → resnet18 + Adadelta (BS=8)

Best mIoU: 0.0245 → resnet18 + Adam (BS=8)

Fastest: 640.12s/epoch → resnet18 + SGD+Nesterov (BS=8)

BEST OPTIMIZER PER MODEL

MobileNetV3: SGD+Nesterov → Acc: 90.80% | mIoU: 0.0153 | Time: 667.62s

ResNet-18: Adadelta → Acc: 98.20% | mIoU: 0.0147 | Time: 668.74s

Adam Family vs MTAdamV2 Comparison (Batch Size 8)

Model	Optimizer	Val Acc	Val mIoU	Time/Epoch	LR
MobileNetV3	Adam	81.0%	0.0143	738s	0.001
MobileNetV3	AdamW	84.0%	0.0152	715s	0.001
MobileNetV3	MTAdamV2	83.4%	0.0113	783s	0.001
ResNet-18	Adam	81.7%	0.0245	797s	0.0001
ResNet-18	AdamW	79.1%	0.0204	746s	0.0001
ResNet-18	MTAdamV2	87.6%	0.0140	668s	0.0003

KEY FINDINGS

MobileNetV3:

- Best: AdamW (84.0%) - outperforms MTAdamV2 by +0.6%
- AdamW achieves best mIoU (0.0152)
- MTAdamV2 is slower (783s vs 715s)

ResNet-18:

- Best: MTAdamV2 (87.6%) - outperforms Adam by +5.9%, AdamW by +8.5%
- MTAdamV2 is also fastest (668s vs 797s for Adam)
- Adam achieves best mIoU (0.0245) but lower accuracy

CONCLUSION

✓ MTAdamV2 excels on ResNet-18 (+5.9% over Adam, +8.5% over AdamW)

✓ AdamW better for MobileNetV3 (+0.6% over MTAdamV2)

MTAdamV2's equal task weighting (0.5/0.5) works better for deeper networks (ResNet-18) but standard Adam/AdamW is sufficient for efficient architectures (MobileNetV3).

